CockroachDB: The Resilient Geo-Distributed SQL Database

Taft, Sharif, Matei, et al. — SIGMOD 2020

Sumit Kumar (22CS30056)

Presented for CS60113: Advanced Database Systems

Prerequisite: Raft Consensus (I)

Raft and Replication

Ranges (~64MB) are the unit of replication

Each range is a Raft group
(Raft is a consensus replication protocol)

Default to 3 replicas, though this is configurable

- Important system ranges default to 5 replicas
- Note: 2 replicas doesn't make sense in consensus replication

Cockroach LABS

Before we start: what is Raft?

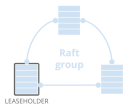
- A consensus protocol that lets a group of replicas agree on an ordered log of commands.
- CockroachDB replicates each ∼64 MB Range as its own independent **Raft group**.
- Default 3 replicas per group — tolerates 1 failure while a majority (2 of 3) stays reachable.

Prerequisite: Raft Consensus (II)

Raft and Replication

Raft provides "atomic replication" of commands

Commands are proposed by the leaseholder replica and distributed to the follower replicas, but only accepted when a quorum of replicas have acknowledged receipt



* Leaseholder == Raft leader

Cockroach LABS

Leader = Leaseholder

- Commands are proposed by the **leader** and replicated to followers.
- A command is only accepted once a **quorum** (majority) of replicas acknowledges it — this is what makes replication "atomic".
- In CockroachDB, the Raft leader for a Range *is* its leaseholder — the term we'll use for the rest of this talk.

Motivation

Why does this matter?

- Modern apps serve users spread across the globe, not one datacenter.
- Regulations like GDPR require certain data to physically stay in-region.
- Users expect low latency *and* an "always on" experience, even during regional outages.
- Legacy single-master DBMSs cannot satisfy all of these at once.

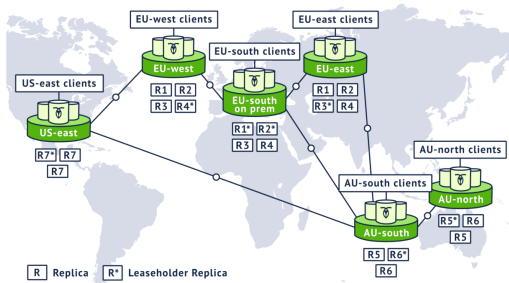


Fig. 1: A global CockroachDB cluster (paper)

Motivation: The Reference Scenario

The business scenario in the paper

- A global company needs EU customer data to stay in the EU (GDPR), while US and AU users get low-latency local access.
- The system must survive a full regional failure without losing consistency.
- Above: EU-west/EU-east/EU-south regions, US-east, and AU-north/AU-south each hold local replicas (`R1`–`R7`); leaseholders (`R*`) are placed near the clients they serve.

Problem Statement

What CockroachDB set out to build

- A SQL database that is **horizontally scalable** on commodity, off-the-shelf hardware.
- Supports **geo-distributed, serializable transactions** spanning multiple partitions.
- Provides **fault tolerance and high availability** without specialized hardware (e.g. atomic clocks).
- Gives users fine-grained control over *where* data physically lives.

Key constraint

- Clock synchronization must rely only on NTP-like software services — no GPS/atomic clocks (unlike Google Spanner's TrueTime).

Background: Architecture Basics

Layered design

- SQL layer: parser, optimizer, executor.
- Transactional KV layer: atomicity & isolation.
- Distribution layer: monolithic ordered key-space.
- Storage layer: local disk-backed KV store (RocksDB).

Data partitioning

- Shared-nothing cluster; any node accepts SQL.
- Data split into $\sim$64 MiB **Ranges** (contiguous key chunks).
- Ranges split/merge automatically as data grows/shrinks or gets hot.

Background: Replication & Leases

Raft consensus per Range

- Each Range is replicated (default 3×) as a **Raft group**.
- Raft elects a leader; writes go through it and commit once a majority acknowledges.

Leaseholder

- One replica per Range holds the **lease** to serve authoritative reads and propose writes.
- Leases avoid a network round trip through Raft for every read.

The Gap in Existing Work (I)

Spanner (Google)

- Achieves strict serializability using **TrueTime** — GPS + atomic clocks bound clock uncertainty to milliseconds.
- Read-write transactions acquire locks and wait out the uncertainty window on every commit.
- Requires specialized hardware $\Rightarrow$ effectively locks you into Google Cloud.

Calvin, FaunaDB, SLOG

- Provide strict serializability but require deterministic execution with reads/writes known up front.
- This breaks support for conversational, ad-hoc SQL.

The Gap in Existing Work (II)

The open question

- Can we get serializable, geo-distributed SQL transactions on *commodity hardware* with *loosely synchronized* clocks?
- Nobody had shown this was possible without either specialized hardware (Spanner) or giving up interactive SQL (Calvin/FaunaDB/SLOG).
- This is precisely the gap CockroachDB fills.

Core Contribution 1: Hybrid-Logical Clocks (HLC)

What is an HLC?

- Combines physical wall-clock time with a Lamport logical counter.
- Every inter-node message carries an HLC timestamp; receivers advance their own clock to stay causally consistent.

Properties

- Causality tracking across nodes.
- Strict monotonicity, even across process restarts.
- Self-stabilization: clocks re-converge after transient skew.

Core Contribution 2: Uncertainty Intervals

The problem with loose clocks

- Without a global clock, a transaction cannot always tell if a value it reads was written "before" or "after" it in real time.

The solution

- Each transaction gets an interval $[commit\_ts,\ commit\_ts + max\_offset]$.
- Any value found inside this window is treated as a potential past write $\Rightarrow$ triggers an **uncertainty restart** that bumps the timestamp forward.
- Guarantees single-key linearizability *without* a hardware clock bound.

Behavior Under Clock Skew

What if clocks drift beyond `max_offset`?

- Within a Range, Raft never depends on clocks — ordering stays correct regardless of skew.
- **Safeguard 1**: lease intervals are disjoint, so two leaseholders can never be simultaneously active.
- **Safeguard 2**: every write carries the lease sequence number; a stale lease's writes are rejected.

Self-protection

- If a node's clock drifts more than 80% of `max_offset` from the cluster, it self-terminates.
- Worst case under skew: a rare stale read (linearizability), never a broken transaction.

Core Contribution 3: Optimistic Concurrency Control

MVCC with write intents

- Every transaction writes provisional values called **intents**, pointing to a transaction record (pending/staging/committed/aborted).
- Readers that encounter an intent check its transaction record to resolve the value's true status.

Conflict resolution

- Write-read, read-write, and write-write conflicts are resolved by *advancing the transaction's timestamp* rather than blocking outright.
- A distributed deadlock detector aborts one transaction in a wait-cycle.

Optimization 1: Read Refreshes

Avoiding unnecessary restarts

- When a transaction's timestamp must advance, CRDB tries a **read refresh** first: re-check that previously read keys haven't changed in the new interval.
- If nothing changed, the transaction continues forward at the new timestamp — no expensive restart needed.
- Equivalent in spirit to PostgreSQL's rw-antidependency tracking for SSI.

Analogy

- Like re-checking your shopping list before checkout instead of restarting the whole shopping trip.

Optimization 2: Write Pipelining & Parallel Commits

Write Pipelining

- Return control before replication of the current op finishes.
- Independent (non-overlapping) ops proceed concurrently.

Parallel Commits

- Introduces a **staging** status: commit is conditional on all writes finishing replication.
- Replicates the commit marker *in parallel* with outstanding writes, saving a round of consensus.

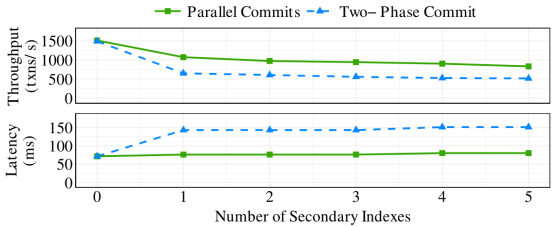


Figure 2: Performance impact of Parallel Commits

Fig. 2: up to 72% higher throughput, 47% lower p50 latency

Optimization 3: Geo-Partitioning & Follower Reads

Data placement policies

- **Geo-Partitioned Replicas**: pin a partition's Ranges to one region (fast local reads/writes, region-scoped failure domain).
- **Geo-Partitioned Leaseholders**: pin leaseholders to the primary region, replicas elsewhere (survives regional failure, slower cross-region writes).
- **Duplicated Indexes**: replicate a read-mostly index into every region for fast local reads.

Follower Reads

- Non-leaseholder replicas can serve historical (`AS OF SYSTEM TIME`) reads once they know no future writes can invalidate them, via a **closed timestamp**.

The SQL Layer & Query Optimizer

Cascades-style optimizer

- Explores over 200 transformation rules to find a low-cost query plan.
- Rules are written in **Optgen**, a DSL that compiles to Go for match/replace transformations.
- Normalization and Exploration rules are interleaved, like the classic Cascades framework.

Distribution-aware planning

- Uses table partitioning to infer extra filters and enable more selective index scans (like Oracle's index skip scan, but static).
- For duplicated indexes, costs each replica by distance to the query's gateway node — minimizing cross-region shuffling.

Physical Planning & Execution

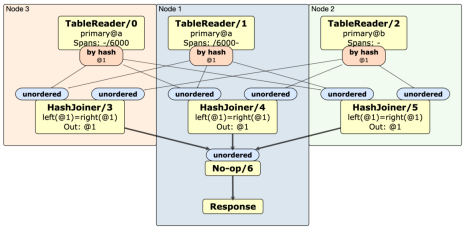


Figure 3: Physical plan for a distributed hash join

From logical plan to physical DAG

- Logical scans split into per-node **TableReader** operators.
- Remaining operators (joins, filters, aggregations) are pushed down to run on the same nodes as the data.
- Distributed only when a heuristic estimates enough data would otherwise cross the network.

Execution Engines: Row-at-a-Time vs. Vectorized

Row-at-a-time (Volcano-based)

- CockroachDB's primary engine; supports every SQL feature (joins, window functions, etc.).
- Processes one row at a time through the iterator tree — simple, but interpreter overhead adds up.

Vectorized engine

- Inspired by MonetDB/X100; operates on column-oriented batches instead of rows.
- Operators are monomorphized per data type (Go lacks generics), avoiding interpreter overhead.
- Up to 2 orders of magnitude faster per-operator; up to 4× on TPC-H queries.

Schema Changes

Staying online during schema changes

- Tables must remain readable and writable while a schema change (e.g. adding an index) is in progress.
- Different nodes may be running different schema versions during the rollout.
- CRDB follows F1's approach: decompose each change into a sequence of incremental versions.
- Adding a secondary index needs two intermediate versions, ensuring it is fully updated on writes *before* it becomes readable — keeping at most two schema versions live cluster-wide at once.

Experimental Setup

Benchmarks used

- **Sysbench OLTP**: vertical/horizontal scalability (3–48 nodes, up to 36 vCPUs/node).
- **TPC-C**: cross-node coordination cost, and a 100,000-warehouse (8 TB) scale test vs. Amazon Aurora.
- **YCSB**: head-to-head throughput/latency comparison against Google Cloud Spanner.

Multi-region fault injection

- 9-node, 3-region GCP cluster; AZ failure and full region failure injected mid-run to compare the four data-placement policies.

Results (I): Scalability

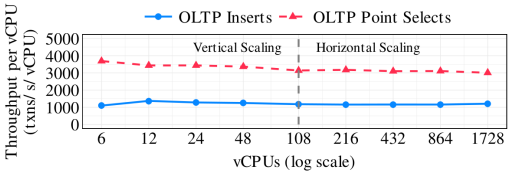


Figure 4: Maximum throughput per vCPU for Sysbench workloads with varying number of vCPUs

Fig. 4: throughput per vCPU stays flat, 3–48 nodes

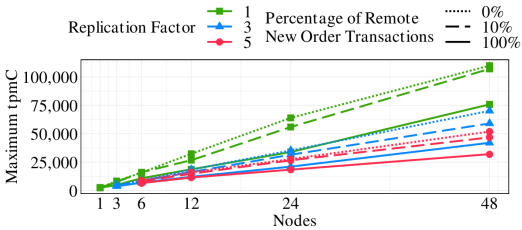


Figure 5: Maximum tpmC with varying % of remote transactions, cluster sizes, and replication factors

Fig. 5: cross-node coordination cost on TPC-C

Takeaway

- Near-linear horizontal scaling; remote transactions cost up to 46–57% throughput, but the scaling trend still holds.

Results (I): TPC-C at Scale vs. Amazon Aurora

	Warehouses		
	1,000	10,000	100,000
CockroachDB Max tpmC	12,474	124,036	1,245,462
CockroachDB Efficiency	97.0%	96.5%	98.8%
Amazon Aurora Max tpmC	12,582	9,406	—
Amazon Aurora Efficiency	97.8%	7.3%	—

Takeaway

- At 100,000 warehouses: 50 billion rows, 8 TB of data, 81 nodes, 98.8% efficiency.
- Aurora's single-master design collapses to 7.3% efficiency at just 10,000 warehouses.

Results (II): CockroachDB vs. Spanner

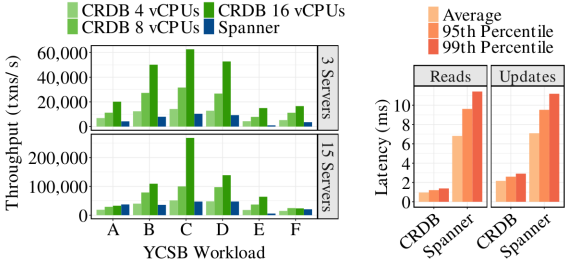


Figure 7: Throughput of CRDB and Spanner on YCSB

Fig. 7: YCSB throughput and latency, CRDB vs. Spanner

Takeaway

- CRDB shows higher throughput on most YCSB workloads and lower latency at all percentiles under light load.
- Exception: write-heavy, highly-contended Workload A, where CRDB's optimistic protocol scales worse.

Results (II): Fault Tolerance Under Failures

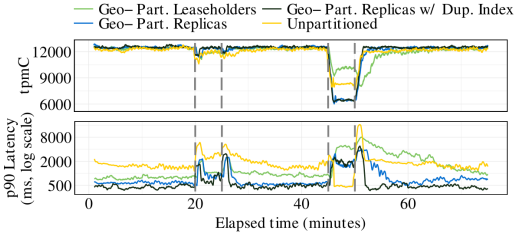


Figure 6: Multi-region cluster performance with various placement policies and AZ/region failures

Fig. 6: TPC-C under AZ and region failures

Takeaway

- All 4 placement policies tolerate an AZ failure with only mild degradation.
- Only **Geo-Partitioned Leaseholders** keeps serving through a full region failure — at the cost of higher steady-state p90 latency.

Case Studies

Virtual customer support agent (telecom)

- 24/7 agent backed by a sessions database recording conversation metadata.
- Hybrid on-prem + AWS multi-region deployment; chose CRDB for strong consistency and regional fault tolerance.

Global platform for an online gaming company

- 30–40 million financial transactions/day; core users in Europe and Australia, fast-growing US base.
- Needed data compliance, consistency, performance, and availability together — adopted geo-partitioned leaseholders.

Lessons Learned

Operational hindsight from 5 years in production

- **Raft in practice**: heartbeat chatter had to be coalesced; adopted Joint Consensus for safe membership changes during rebalancing.
- **Snapshot isolation removed**: supporting it safely alongside SERIALIZABLE needed pessimistic locking everywhere — not worth the complexity.
- **Postgres compatibility** boosted adoption but differing retry semantics remain a friction point.
- **Version upgrades**: near-zero-downtime rolling upgrades are hard — mixed-version clusters add real complexity.

Conclusions & Takeaways

What CockroachDB proves

- Serializable isolation at global scale is achievable *without* specialized clock hardware.
- Parallel Commits + read refreshes recover most of the latency lost to distributed consensus.
- Configurable data placement lets one system serve very different compliance/latency/fault-tolerance trade-offs.

Open directions (per the authors)

- Disaggregated storage, serverless on-demand scaling, usage-based pricing.
- Geo-aware query optimization remains an active research problem.

Appendix: Live Demonstration Walkthrough

How CockroachDB actually works, visually

- The following slides are adapted from Peter Mattis (Cockroach Labs co-founder), *"Architecture of a Geo-Distributed SQL Database"*, QCon.
- They walk through the exact mechanisms just discussed — Ranges, Raft, leases, geo-partitioning, a live distributed transaction, and SQL-to-KV mapping — with concrete, worked examples.
- Intended as backup / demo material for Q&A, time permitting.

Demo: Monolithic Key Space

Monolithic Key Space

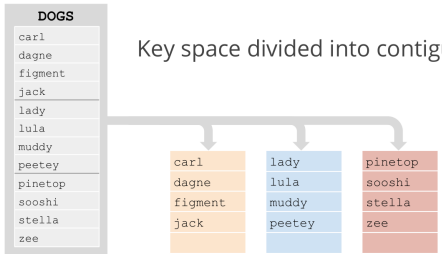
Monolithic logical key space

- Ordered lexicographically by key

Cockroach LABS

Demo: Ranges

Ranges



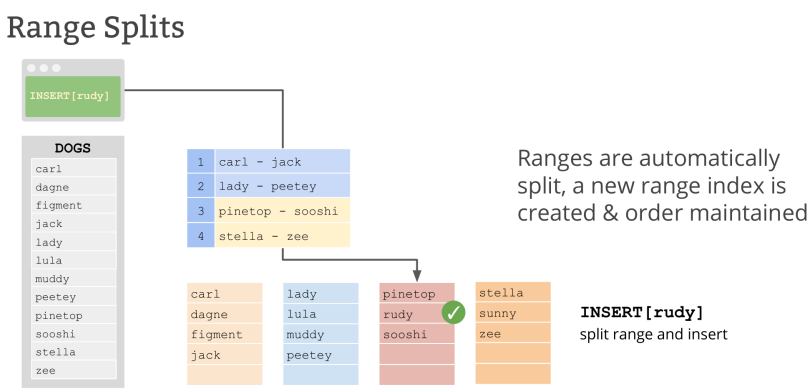
Key space divided into contiguous ~64MB ranges

Ranges are small enough to be moved/split quickly

Ranges are large enough to amortize indexing overhead

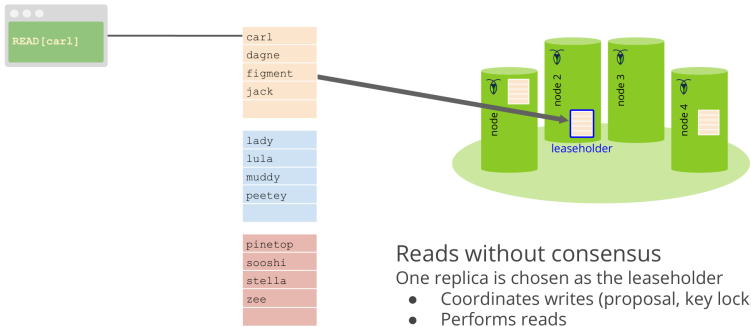
Cockroach LABS

Demo: Range Splits



Demo: Range Leases

Range Leases



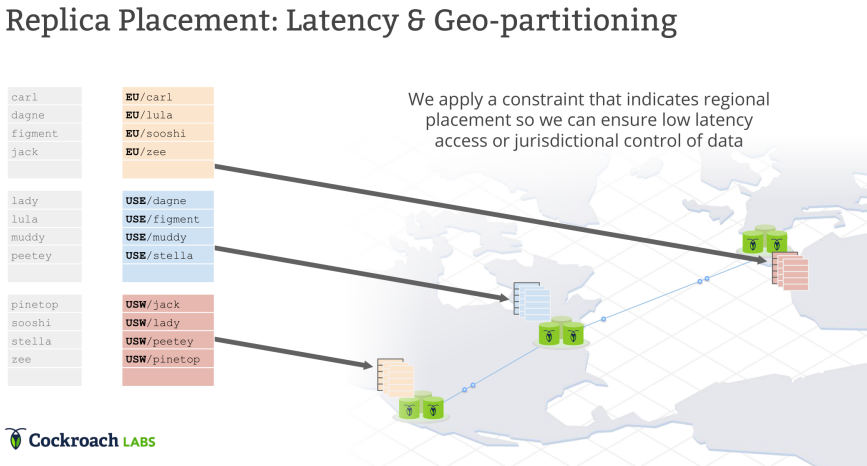
Reads without consensus

One replica is chosen as the leaseholder

- Coordinates writes (proposal, key locking)
- Performs reads

Cockroach LABS

Demo: Geo-Partitioning for Latency & Compliance



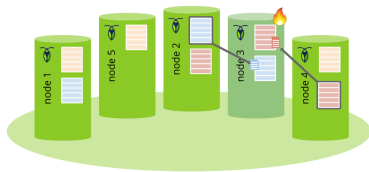
Demo: Rebalancing After a Node Failure

Rebalancing Replicas

Loss of a node

Temporary Failure

If a node goes down for a moment, the leaseholder can “catch up” any replica that is behind



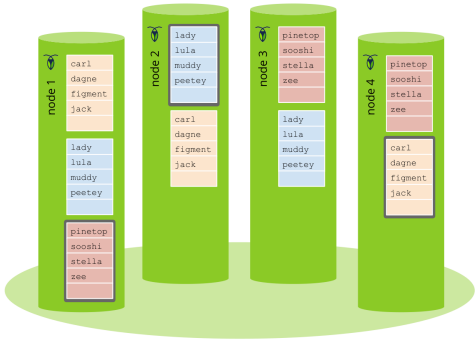
The leaseholder can send commands to be replayed OR it can send a snapshot of the current Range data. We apply heuristics to decide which is most efficient for a given failure.

Cockroach LABS

Demo: A Distributed Transaction (I) — Begin

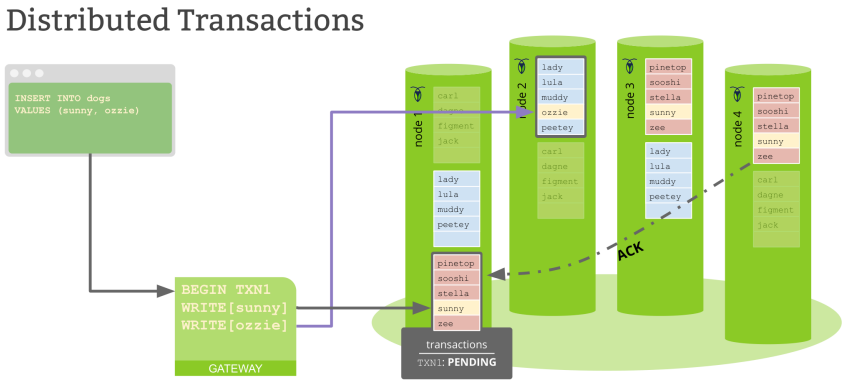
Distributed Transactions

```
INSERT INTO dogs
VALUES (sunny, ozzie)
```

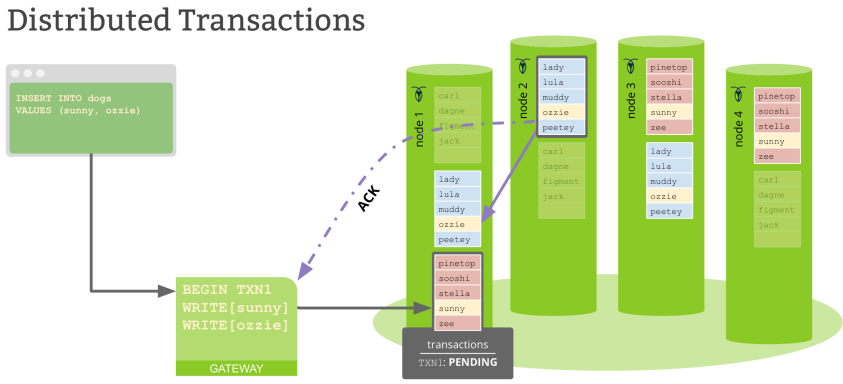


Cockroach LABS

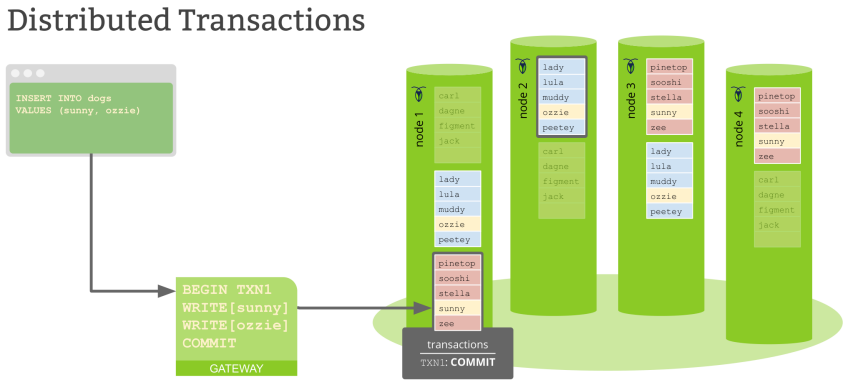
Demo: A Distributed Transaction (II) — First Write



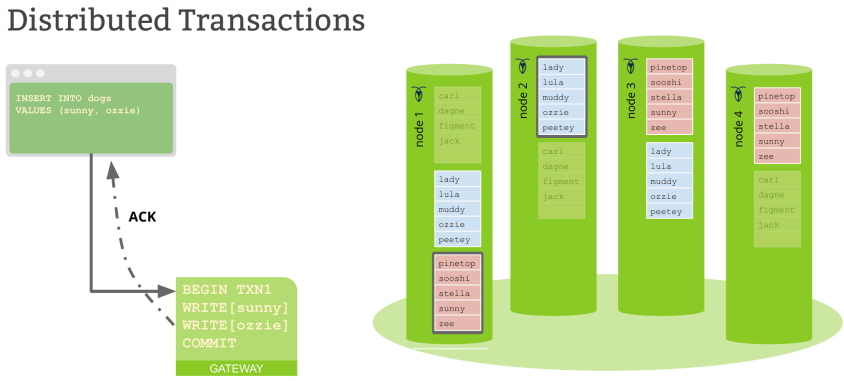
Demo: A Distributed Transaction (III) — Second Write, ACK



Demo: A Distributed Transaction (IV) — Commit



Demo: A Distributed Transaction (V) — Client Acknowledged



Demo: Mapping SQL Tables to Key-Value Pairs

SQL Data Mapping: Inventory Table

```
CREATE TABLE inventory (
     id INT PRIMARY KEY,
     name STRING,
     price FLOAT,
     INDEX name_idx (name)
)
```

ID	Name	Price
1	Bat	1.11
2	Ball	2.22
3	Glove	3.33
4	Bat	4.44

Key	Value
/inventory/name_idx/”Bat”/1	∅
/inventory/name_idx/”Ball”/2	∅
/inventory/name_idx/”Glove”/3	∅
/inventory/name_idx/”Bat”/4	∅

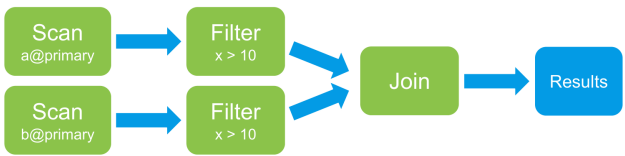
Cockroach LABS

Demo: Query Optimization — Filter Push-Down

SQL Optimization: Filter Push-Down

```
SELECT * FROM a JOIN b WHERE x > 10
```

After filter push-down



Cockroach LABS

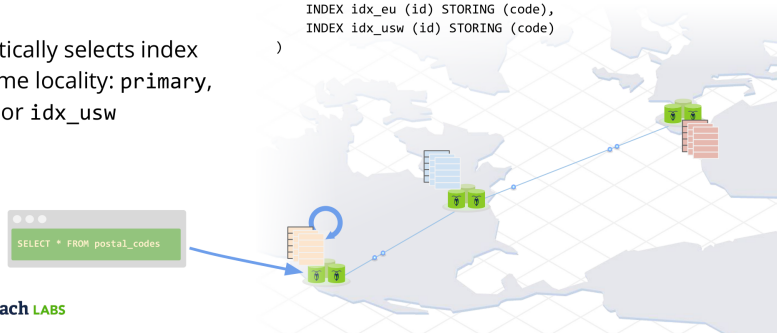
Demo: Locality-Aware Query Optimization

Locality-Aware SQL Optimization

Optimizer includes locality in cost model

Automatically selects index from same locality: `primary`, `idx_eu`, or `idx_usw`

```
CREATE TABLE postal_codes (
    id INT PRIMARY KEY,
    code STRING,
    INDEX idx_eu (id) STORING (code),
    INDEX idx_usw (id) STORING (code)
)
```



Cockroach LABS

Thank You

Questions?

Sumit Kumar (22CS30056)

CS60113: Advanced Database Systems